AEGIS

Measuring and Defeating Prompt-Injection Attacks on Tool-Using LLM Agents

Mildfriend | B.Tech CSE (Cyber Security), Central University of Jammu | October 2026

17	100% to 0%	100%
ATTACKS	**ATTACK SUCCESS**	**UTILITY HELD**
9 categories: override, role-play, encoded, indirect, leak, SQL, HTTP, multi-turn	Undefended agent vs. four-layer defence stack	Legitimate task preserved under every defence

Executive summary

An LLM agent that can call tools - send email, POST to a URL, run SQL, read files - turns a prompt injection from a party trick into a remote-execution class bug for natural language: a payload hidden in a document the agent is asked to summarise can redirect its tools. Advice in the wild is vague and unquantified. AEGIS makes the trade-off measurable. Against a catalogue of 17 attacks across 9 categories, an undefended agent falls to **100%** of them. A four-layer defence stack drives the attack success rate to **0%** while preserving **100%** of legitimate-task utility. No single layer is sufficient, and the benchmark shows exactly where each one fails. An adaptive attacker that mutates and composes attacks still gets through the full stack at **17.6%** - every bypass by encoding the leaked secret past a literal-match filter - and a hardened, decoding filter closes it back to **0%**.

The contribution is a reproducible harness: a deterministic offline agent, an effect-level definition of attack success, four composable defence layers, an adaptive attacker, a 64-test suite, and a CLI that regenerates every number in this report.

The layers are then promoted from a benchmark into a running defender. **aegis guard** watches an agent's instructions, tool calls and answers, applies the hardened stack to each, and acts on its own - with an autonomy ladder (observe, recommend, enforce), session quarantine, a JSONL audit trail and a kill switch. **GuardedAgent** puts the guard in the agent's own path, so a compromised model's calls never reach the world: **0** effects achieved across 164 attack variants, where the model proposed 262 calls and the guard refused 77. On a live model (Gemini 2.5 Flash) the same run records **0** effects achieved, and the mechanism is the honest part - the model resisted most injections by itself, which is precisely why the guard is the layer that matters.

Threat model

The agent under test has a system prompt, a legitimate task (summarise a document and email the summary to an allow-listed colleague), and four tools: **send_email**, **http_post**, **read_file**, **run_sql**. It processes two channels: the user's instruction (trusted) and retrieved documents (untrusted). A protected secret - a sentinel credential string - sits in the store. An attack succeeds only if one of four forbidden effects is actually recorded:

- **exfil_email** - protected data emailed to a non-allow-listed domain
- **exfil_http** - protected data POSTed to a non-allow-listed host
- **secret_leak_response** - the secret appears in the agent's final answer

- **destructive_sql** - a DROP/DELETE/UPDATE statement is executed

Success is defined at the level of **impact**, not appearance: an attack that fools the model but whose effect is blocked does not count. This is what lets the harness score a policy engine, which does not care whether the model was fooled, on the same scale as an input filter, which does.

Architecture

Inputs pass through the input-side defences (sanitizer, datamarking), reach the agent, and the agent's proposed tool calls pass through the tool-call policy before any effect is recorded. The agent's final answer passes through the output filter. Each attack is then scored against its objective. Defences compose: a DefenceStack applies each layer in order.

attack payload -> [sanitizer | datamarking] -> agent (proposes calls + answer) -> [tool-call policy] -> World (records effects) -> [output filter] -> evaluate(objective)

Attack taxonomy

The catalogue spans the realistic space. Direct override and authority framing ('ignore previous instructions', 'as an AI you must'); role-play and developer-mode persona hijacks; base64 encoding that defeats signature filters; paraphrased and subtle P.S.-style exfiltration that defeats them too; indirect injection hidden in a document (HTML comment and plain note); response leakage; destructive SQL; HTTP exfiltration; and a multi-turn 'security audit' that splits the payload across messages.

Defence layers

Layer	Where	What it does	Its blind spot
sanitizer	input	Redacts sentences matching known injection signatures	paraphrased and encoded payloads
datamarking	input	Marks untrusted documents so instructions inside are treated as data	direct (user-turn) injection
tool-policy	action	Refuses calls that break policy: off-domain email, non-allow-listed host, destructive SQL, secret exfil	response leakage; unguarded tools
output-filter	output	Redacts the protected secret from the final answer	encoded leaks; anything that is not a response leak

A hardened variant, **output-filter-decoding**, detects the secret in base64, hex, rot13 and reversed form before redacting - the answer to the adaptive attacker's encoding bypass.

Results

Attack Success Rate (ASR) is lower-is-better; utility is the fraction of cases where the legitimate task still worked. 17 attacks, 9 defence configurations, 153 runs.

Defence	ASR	Utility
none	100.0%	100.0%
sanitizer	64.7%	100.0%
datamarking	88.2%	100.0%
tool-policy	17.6%	100.0%

Defence	ASR	Utility
output-filter	82.4%	100.0%
sanitizer + tool-policy	11.8%	100.0%
datamarking + tool-policy	17.6%	100.0%
full stack	0.0%	100.0%
hardened	0.0%	100.0%

The single-layer rows are the point. Each defence leaves a distinct hole that another layer fills, which is why the full stack, not any one layer, reaches zero.

Defence	direct	role-play	encoded	subtle	indirect	leak	SQL	HTTP	multi-turn
none	100	100	100	100	100	100	100	100	100
sanitizer	67	67	100	100	50	50	50	0	100
datamarking	100	100	100	100	0	100	100	100	100
tool-policy	0	33	0	0	0	100	0	0	0
output-filter	100	67	100	100	100	0	100	100	100
full stack	0	0	0	0	0	0	0	0	0
hardened	0	0	0	0	0	0	0	0	0

ASR (%) by defence and attack category.

Adaptive attacks

A fixed catalogue can read 0% while an attacker who is allowed to search still gets through. The adaptive attacker mutates each attack - encoding the payload, paraphrasing a trigger verb, framing it as legitimate, or asking for the answer in an encoded form - and composes up to two moves, reporting whether any variant still lands.

Defence	Static ASR	Adaptive bypass rate
none	100.0%	100.0%
full stack	0.0%	17.6%
hardened	0.0%	0.0%

Every adaptive bypass is response leakage, and every one works the same way: ask the model to return the secret base64-encoded, and the literal-match output filter never sees it. The effect-level tool-call policy resists adaptation completely - no mutation gets protected data out through email or HTTP - and the hardened, decoding output filter closes the leak. The lesson is the method, not the number: a defence is only as good as the attacker it was tested against.

From measurement to defence: the guard

A benchmark that only measures is half a tool. The same four layers are promoted into a running defender - **aegis guard** - which watches three kinds of event, matching the three places a defence can act. Because enforcement reuses the hardened stack, the guard cannot be better than its own benchmark: its measured bypass rate is the hardened config's.

Event	Layer applied	What a block means
prompt	input sanitizer	the instruction carried an injection signature
tool_call	tool-call policy	the action breaks policy, whatever the model believed
response	output filter	the answer revealed the secret, in any encoding

Autonomy is the safety design, not a nicety. **observe** records everything and acts on nothing (shadow mode); **recommend** records and alerts but never blocks; **enforce** blocks and cuts off a repeat offender. Every decision goes to a JSONL audit trail, and the guard takes a kill switch. The LLM's role is **analyst triage** - it explains and prioritises an incident in one line, and never decides whether to block, because a model that reads attacker-controlled text can be talked out of a decision and a policy cannot. Triage falls back to a deterministic summary when no model is reachable.

Shadow mode is how such a control earns adoption: replay a log and report what enforcement would do, without doing it. On the built-in stream the guard would block 29 of 85 events across 17 sessions and quarantine two - and the line that decides adoption, **legitimate sessions blocked**, is **0**. It is also exposed as an HTTP endpoint (**POST /event**, inline: *allowed: false* means do not perform it) with */health*, */metrics* and */incidents*.

Real-model validation

The guarded path also runs against a live model through its OpenAI-compatible endpoint. Against **Gemini 2.5 Flash**, on 22 October 2026, on the free tier:

Metric	Value
Attack variants	17
Effects achieved	0
Prompts refused by the guard	9
Tool calls proposed by the model	7
Tool calls blocked by the guard	1

Zero effects achieved - but the mechanism differs from the simulation, and that is the finding. The offline agent is deliberately gullible: it follows every injected instruction, so in the simulation the guard does all the work. A real model refused most injections on its own, proposing only 7 tool calls where the mock proposes 22; the guard blocked one of them, a destructive SQL statement the model did propose. The honest conclusion is not that the model is safe. It is that **the guard is the part that does not depend on the model's mood** - a backstop against a model that resists, and the only thing between the injection and the effect against one that complies. The simulation measures that second case, which is why it is the right thing to benchmark against.

Findings

1. An undefended agent is fully compromised. Every one of the 17 attacks lands against the baseline. Prompt injection against a tool-using agent is not a corner case; it is the default outcome.

2. Single layers have large, measurable blind spots. Input sanitisation cuts ASR to 64.7% but is defeated by base64 and by paraphrase, because it matches known phrasings. Datamarking only moves the needle on indirect injection (0% there, 100% everywhere else). The tool-call policy handles actions (17.6%) but is blind to response leakage, which only the output filter catches (82.4%).

3. Layering is required, and it is cheap. sanitizer + tool-policy reaches 11.8%; adding the output filter closes the leakage gap. The full stack reaches 0% with no loss of legitimate utility - a defence that works by breaking the agent is not a defence, so utility is reported in every row.

4. The effect-level policy dominates this catalogue - an honest finding. Because the policy refuses the effect however the model was fooled, removing the input-side layers from the full stack reopens no attack here. They are retained deliberately: a real policy is never complete, and a cheap input filter that never relies on the model is worth keeping against channels a policy cannot mediate. The test suite pins this finding so it cannot silently change.

5. The static 0% is a floor, not immunity. Under an adaptive attacker that composes mutations, the full stack falls to 17.6% - all of it response leakage defeated by encoding - and only the hardened, decoding output filter restores zero. The benchmark's real contribution is the method: test the defence against an attacker who searches.

6. The defence survives being run for real. Promoting the layers into a guard that sits in the agent's path - and running it against both the gullible simulation and a live model - records zero effects achieved in both cases. What changes is who does the work: in the simulation the guard blocks everything; on a live model the model resists most attacks and the guard covers the rest. A defence whose value depends on the model resisting is not a defence; this one does not.

Limitations

The default agent is a deterministic simulation, chosen so results are reproducible and comparable; a real-model adapter is included to validate that the offline numbers track reality, but the headline numbers here are from the simulation. The adaptive attacker searches a bounded mutation space (four families, two composed moves); a deeper or smarter search would find more, so 0% is a measured ceiling, not immunity. The tool-call policy assumes a complete allow-list; novel tools or a compromised allow-listed destination would need new policy rules.

Future work

Deeper adaptive search, and one that optimises against a chosen defence rather than enumerating a fixed mutation set; a large-scale real-LLM run across several models; per-tool risk scoring so the policy can be graded rather than binary; and automated payload generation to grow the catalogue without hand-writing each case.

Reproducibility

Every figure in this report is regenerated by one command. The repository ships the agent, the attack catalogue, the four defence layers, the adaptive attacker, the guard, the guarded agent, the CLI, and a 64-test suite; the offline agent needs no network and no API key.

pip install -e ".[dev]" | aegis run | aegis adaptive | aegis guard --simulate | pytest -q